

Neon Postgres Migration Guide

This guide explains how to move the Solaris Health database to [Neon](https://neon.tech) (https://neon.tech) (serverless Postgres) **without changing any application code**. The backend reads its connection string exclusively from the `DATABASE_URL` environment variable, so migrating providers is purely a configuration change.

Note: This is an optional portability path. The current production deployment continues to run on its existing Postgres instance. Do **not** run these steps against the live database unless you are intentionally cutting over to Neon.

1. Prerequisites

- A [Neon account](https://console.neon.tech) (https://console.neon.tech) (free tier is sufficient to start).
 - `pg_dump` / `psql` v15+ installed locally (matches the production `postgres:15-alpine` image). Verify with `psql --version`.
 - Access to the **current** `DATABASE_URL` (source) — never commit it.
-

2. Create a Neon project

1. Log in to the Neon console and click **New Project**.
2. Choose Postgres **15** to match the current major version.
3. Pick the region closest to where the backend is hosted (lowest latency).
4. After creation, open **Connection Details** and copy the pooled connection string. It looks like:

```
postgresql://<user>:<password>@<endpoint>.neon.tech/<db>?sslmode=require
```

The `sslmode=require` suffix is important — the backend's DB layer (`backend/src/db.js`) automatically enables TLS when the connection string contains `sslmode=require` (or when `PGSSL=true` is set), so no code change is needed.

3. Dump the current database

Export the full schema **and** data from the source database:

```
# SOURCE_URL = the current production DATABASE_URL
pg_dump "$SOURCE_URL" \
  --no-owner \
  --no-privileges \
  --format=plain \
  --file=solaris_dump.sql
```

- `--no-owner` / `--no-privileges` strip role/ownership statements so the dump restores cleanly under Neon's managed role.
- Use `--format=plain` for a portable, inspectable SQL file.

For a data-only or schema-only export (rarely needed), add `--data-only` or `--schema-only`.

4. Restore into Neon

```
# NEON_URL = the Neon connection string from step 2
psql "$NEON_URL" --file=solaris_dump.sql
```

Watch the output for errors. Because the schema uses `CREATE TABLE IF NOT EXISTS` and guarded inserts, re-running is safe.

Verify the restore

```
psql "$NEON_URL" -c "\dt" # list tables
psql "$NEON_URL" -c "SELECT count(*) FROM users;"
psql "$NEON_URL" -c "SELECT version();" # confirm PG 15
```

You should see all application tables (users, bookings, health_documents, wallet tables, messaging tables, marketplace tables, etc.) plus the `pgmigrations` bookkeeping table used by node-pg-migrate.

5. Point the app at Neon

Update the `DATABASE_URL` environment variable in your hosting provider — the application requires **no code change**.

Railway

- Project → **Variables** → set `DATABASE_URL` to the Neon string.
- Redeploy (Railway auto-redeploys on variable change).

Render

- `render.yaml` provisions a managed `solaris-db` by default. To use Neon instead, remove/override the `fromDatabase` reference for `DATABASE_URL` in the dashboard and paste the Neon string as a plain env var, then redeploy.

Heroku

```
heroku config:set DATABASE_URL="$NEON_URL" -a <your-app>
```

Docker / self-hosted

- Set `DATABASE_URL` (and optionally `PGSSL=true`) in the container environment. Remove the bundled `postgres` service from `docker-compose.yml` if you no longer need the local DB.

6. Run migrations against Neon (optional)

The backend runs `npm run migrate` automatically on startup, so migrations are applied on first boot. To pre-apply them manually:

```
cd backend
DATABASE_URL="$NEON_URL" npm run migrate
```

`node-pg-migrate` records applied migrations in the `pgmigrations` table and is idempotent — already-applied migrations are skipped.

7. Smoke-test the cutover

After the app is pointed at Neon and redeployed:

```
# Health endpoint (checks DB connectivity)
curl -s https://<your-domain>/api/health | jq

# Demo login
curl -s -X POST https://<your-domain>/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"sofia@solaris.health","password":"demo123"}' | jq
```

Expect `status: "ok"` with `database: "ok"` from `/api/health`, and a JWT token from the login call.

8. Rollback

If anything looks wrong, set `DATABASE_URL` back to the original Postgres connection string and redeploy. Because the Neon migration is additive (a copy, not a move), the original database is untouched and remains the source of truth until you deliberately decommission it.

Why this works with zero code changes

- `backend/src/db.js` builds its pool **solely** from `DATABASE_URL` — there are no hardcoded hosts, users, or passwords.
- TLS is auto-negotiated from the connection string (`sslmode=require`) or the `PGSSL` flag, so Neon's required SSL "just works."
- Schema and migrations are provider-agnostic standard SQL.

This is the 12-factor "backing services as attached resources" principle: the database is swappable via configuration alone.